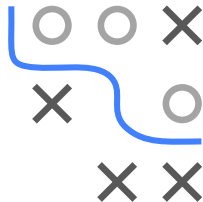


Optimization in Machine Learning

Classic first order methods

Gradient descent



Learning goals

- Iterative Descent / Line Search
- Descent directions
- Gradient descent
- Pseudo-residuals
- ERM with GD

ITERATIVE DESCENT

- **Goal:** For $f : \mathbb{R}^d \rightarrow \mathbb{R}$ continuously differentiable, solve

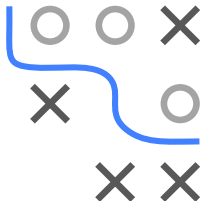
$$\arg \min_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x})$$

- Usually no closed-form solution; construct iterates $\mathbf{x}^{[0]}, \mathbf{x}^{[1]}, \dots$ using only **local** information $f(\mathbf{x}), \nabla f(\mathbf{x})$
- Metaphor: f = height of a mountain at coordinates (x_1, x_2) ; in dense fog, we cannot see the valley, only sense the local slope
- **Central idea:** Repeat

- ❶ At current $\mathbf{x} \in \mathbb{R}^d$, find a **descent direction** $\mathbf{d} \in \mathbb{R}^d$
- ❷ Update $\mathbf{x} \leftarrow \mathbf{x} + \alpha \mathbf{d}$, with step size α s.t. f sufficiently decreases (**step size control**)



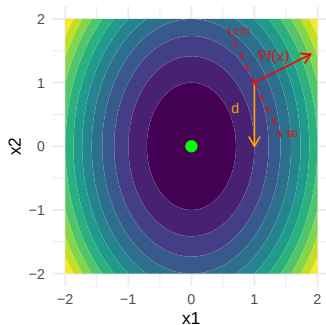
“Walking down the hill through local steps, towards the valley”



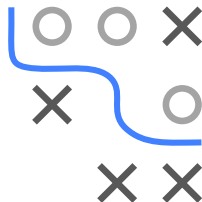
ITERATIVE DESCENT

- **Definition:** $\mathbf{d} \in \mathbb{R}^d$ is a **descent direction** in $\mathbf{x}$ if

$$D_{\mathbf{d}}f(\mathbf{x}) = \nabla f(\mathbf{x})^T \mathbf{d} < 0 \text{ (neg. directional derivative)}$$



Angle between $\nabla f(\mathbf{x})$ and $\mathbf{d}$ must be $> 90^\circ$.



ITERATIVE DESCENT

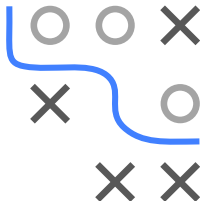
Algorithm Iterative Descent / Line search

- 1: Starting point $\mathbf{x}^{[0]} \in \mathbb{R}^d$
 - 2: **while** Stopping criterion not met **do**
 - 3: Calculate a descent direction $\mathbf{d}^{[t]}$ for current $\mathbf{x}^{[t]}$
 - 4: Find $\alpha^{[t]}$ s.t. $f(\mathbf{x}^{[t]} + \alpha^{[t]}\mathbf{d}^{[t]}) < f(\mathbf{x}^{[t]})$
 - 5: Update $\mathbf{x}^{[t+1]} = \mathbf{x}^{[t]} + \alpha^{[t]}\mathbf{d}^{[t]}$
 - 6: **end while**
-

Key questions:

- How to choose $\mathbf{d}^{[t]}$ (now)
- How to choose $\alpha^{[t]}$ (later)

NB: Above procedure also called **line search**, but term is ambiguous: can serve as umbrella term for the whole scheme or refer only to Step 4.



GRADIENT DESCENT

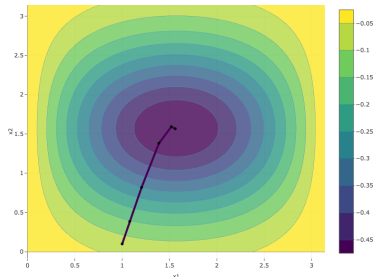
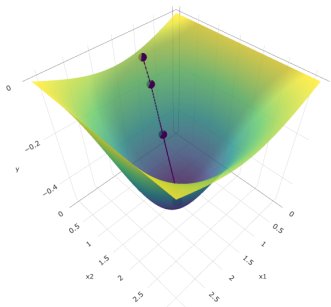
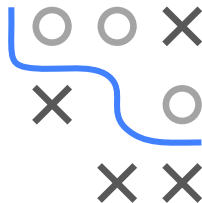
- Locally, $\nabla f(\mathbf{x})$ is the direction of steepest ascent, $-\nabla f(\mathbf{x})$ of steepest descent

- Gradient descent (GD):** iterative descent with $\mathbf{d} = -\nabla f(\mathbf{x})$:

$$\mathbf{x}^{[t+1]} = \mathbf{x}^{[t]} - \alpha^{[t]} \nabla f(\mathbf{x}^{[t]})$$

- Gradient ascent (GA):** maximize f analogously via:

$$\mathbf{x}^{[t+1]} = \mathbf{x}^{[t]} + \alpha^{[t]} \nabla f(\mathbf{x}^{[t]})$$



GD for $f(x_1, x_2) = -\sin(x_1) \cdot \varphi(x_2)$, with φ the density of $\mathcal{N}(\pi/2, 0.8^2)$.

GD FOR EMPIRICAL RISK MINIMIZATION

- Let $\mathcal{D} = ((\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)}))$ and consider ERM:

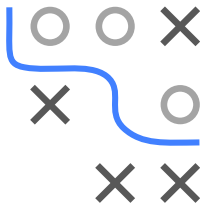
$$\arg \min_{\theta} \mathcal{R}_{\text{emp}}(\theta) = \arg \min_{\theta} \sum_{i=1}^n L(y^{(i)}, f(\mathbf{x}^{(i)} | \theta))$$

- Gradient** of the empirical risk:

$$\nabla_{\theta} \mathcal{R}_{\text{emp}}(\theta) = \sum_{i=1}^n \frac{\partial L(y^{(i)}, f(\mathbf{x}^{(i)} | \theta))}{\partial \theta}$$

- GD update rule:**

$$\theta^{[t+1]} = \theta^{[t]} - \alpha^{[t]} \nabla_{\theta} \mathcal{R}_{\text{emp}}(\theta)|_{\theta^{[t]}}$$



GD FOR ERM: PSEUDO-RESIDUALS

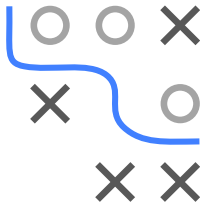
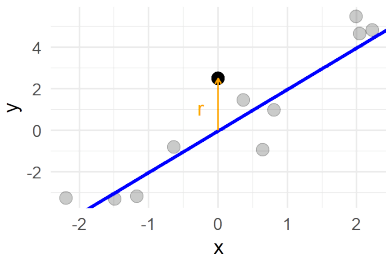
- By the chain rule, the (neg.) gradient of the ERM problem can be expressed as:

$$-\nabla_{\boldsymbol{\theta}} \mathcal{R}_{\text{emp}}(\boldsymbol{\theta}) = \sum_{i=1}^n \underbrace{\left(-\frac{\partial L\left(y^{(i)}, f(\mathbf{x}^{(i)})\right)}{\partial f} \right)}_{\text{pseudo-residual } \tilde{r}^{(i)}(f)} \frac{\partial f(\mathbf{x}^{(i)} | \boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$$

- **Pseudo-residuals** tell us how to distort $f(\mathbf{x}^{(i)})$ to achieve greatest decrease of $L(y^{(i)}, f(\mathbf{x}^{(i)}))$ (best pointwise update)
- $\frac{\partial f(\mathbf{x}^{(i)} | \theta)}{\partial \theta}$ tells us how to modify θ accordingly and wiggle model output
- GD step sums up these modifications across all observations i

NB: Pseudo-residuals $\tilde{r}(f)$ match usual residuals for L2 loss:

$$\begin{aligned}\tilde{r}(f) &= -\frac{\partial L(y, f(\mathbf{x}))}{\partial f} = -\frac{1}{2} \frac{\partial (y-f)^2}{\partial f} \Big|_{f=f(\mathbf{x})} \\ &= y - f(\mathbf{x})\end{aligned}$$



OPTIMIZE LS LINEAR REGRESSION WITH GD

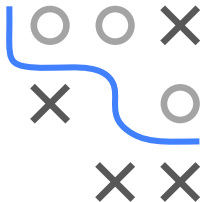
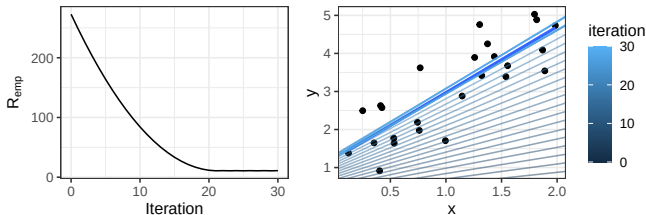
- Let $\mathcal{D} = ((\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)}))$ and minimize

$$\mathcal{R}_{\text{emp}}(\boldsymbol{\theta}) = \sum_{i=1}^n (\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)})^2$$

NB: For illustration, we use GD even though closed-form solution exists. GD-like (more adv.) approaches like this MIGHT make sense for large data, though.

- Gradient:**

$$\nabla_{\boldsymbol{\theta}} \mathcal{R}_{\text{emp}}(\boldsymbol{\theta}) = \frac{\partial \mathcal{R}_{\text{emp}}(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = -2 \sum_{i=1}^n (y^{(i)} - \boldsymbol{\theta}^\top \mathbf{x}^{(i)}) \mathbf{x}^{(i)}$$



GD FOR LOGISTIC REGRESSION

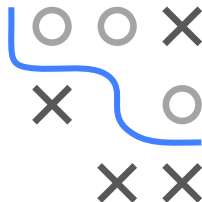
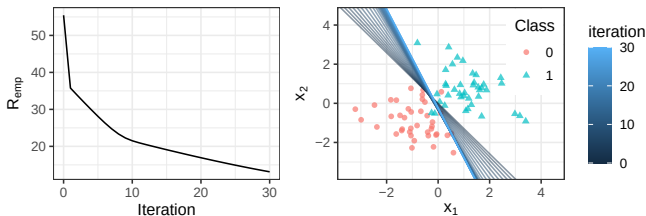
- Let $\mathcal{D} = ((\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)}))$ with $y^{(i)} \in \{0, 1\}$ and minimize the cross-entropy risk

$$\mathcal{R}_{\text{emp}}(\boldsymbol{\theta}) = - \sum_{i=1}^n \left[y^{(i)} \log \pi^{(i)} + (1 - y^{(i)}) \log(1 - \pi^{(i)}) \right]$$

where $\pi^{(i)} := \pi(\mathbf{x}^{(i)} | \boldsymbol{\theta}) = s(\boldsymbol{\theta}^\top \mathbf{x}^{(i)})$, $s(z) = \frac{1}{1 + \exp(-z)}$

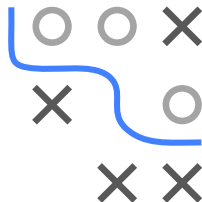
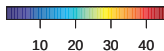
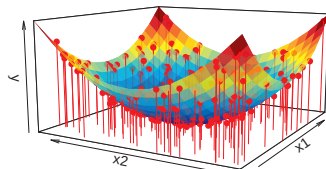
- Gradient:**

$$\nabla_{\boldsymbol{\theta}} \mathcal{R}_{\text{emp}}(\boldsymbol{\theta}) = \sum_{i=1}^n (\pi^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$$



ERM FOR NN WITH GD

- Consider $y = x_1^2 + x_2^2$ and $\mathcal{D} = ((\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)}))$



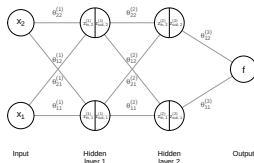
- Fit a neural network $f(\mathbf{x} \mid \boldsymbol{\theta})$ with two hidden layers (16 ReLU units each) by minimizing the empirical risk

$$\mathcal{R}_{\text{emp}}(\boldsymbol{\theta}) = \sum_{i=1}^n \left(f(\mathbf{x} \mid \boldsymbol{\theta}) - y^{(i)} \right)^2$$

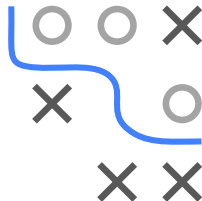
BACKPROPAGATION

- GD requires $\nabla_{\theta} \mathcal{R}_{\text{emp}}(\theta)$; for NNs computed via **backpropagation** (BP)
- Weight θ affects L through **all** paths $\theta \rightarrow z_{\text{in}} \rightarrow z_{\text{out}} \rightarrow \dots \rightarrow f \rightarrow L$ in the network; chain rule: $\frac{\partial L}{\partial \theta} = \text{sum over these paths of products of local derivatives}$
- BP: **forward pass** computes all unit values; **backward pass** accumulates the path sums from output to input, reusing shared factors
- **Example** (2 hidden layers with 2 units each, activation σ): let $L = \frac{1}{2}(y - f(x))^2$ and $z_{\text{in},1}^{(1)} = \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2$, $z_{\text{out},1}^{(1)} = \sigma(z_{\text{in},1}^{(1)})$, etc.

$$\frac{\partial L}{\partial \theta_{11}^{(3)}} = \underbrace{\frac{\partial L}{\partial f}}_{f-y} \cdot \underbrace{\frac{\partial f}{\partial \theta_{11}^{(3)}}}_{z_{\text{out},1}^{(2)}}, \quad \frac{\partial L}{\partial \theta_{11}^{(2)}} = \underbrace{\frac{\partial L}{\partial f}}_{f-y} \cdot \underbrace{\frac{\partial f}{\partial z_{\text{out},1}^{(2)}}}_{\theta_{11}^{(3)}} \cdot \underbrace{\frac{\partial z_{\text{out},1}^{(2)}}{\partial z_{\text{in},1}^{(2)}}}_{\sigma'(z_{\text{in},1}^{(2)})} \cdot \underbrace{\frac{\partial z_{\text{in},1}^{(2)}}{\partial \theta_{11}^{(2)}}}_{z_{\text{out},1}^{(1)}}, \quad \dots$$



Note: Shown weights have a unique path to f (single products); layer-1 weights reach f via both layer-2 units $\Rightarrow$ sum of two products. Biases omitted; actual NN: 16 units/layer.



EXCURSUS: BP ERROR SIGNALS

- Vector notation for hidden layers $l = 1, \dots, m$, with weight matrices $\Theta^{(l)} = (\theta_{jk}^{(l)})_{j,k}$ and σ applied elementwise:

$$\mathbf{z}_{\text{in}}^{(l)} = \Theta^{(l)} \mathbf{z}_{\text{out}}^{(l-1)}, \quad \text{i.e., } z_{\text{in},j}^{(l)} = \sum_k \theta_{jk}^{(l)} z_{\text{out},k}^{(l-1)}, \quad \mathbf{z}_{\text{out}}^{(l)} = \sigma(\mathbf{z}_{\text{in}}^{(l)})$$

- Input, linear output, and L2 loss:

$$\mathbf{z}_{\text{out}}^{(0)} = \mathbf{x}, \quad \mathbf{f} = \boldsymbol{\theta}^{(m+1)\top} \mathbf{z}_{\text{out}}^{(m)}, \quad L = \frac{1}{2} (y - f)^2$$

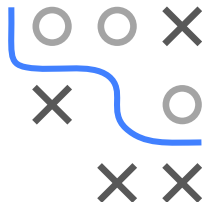
- Error signal** of layer l = sensitivity of L w.r.t. unit inputs:

$$\delta_j^{(l)} := \frac{\partial L}{\partial z_{\text{in},j}^{(l)}}, \quad \text{at the output: } \delta^{(m+1)} = \frac{\partial L}{\partial f} = f - y$$

- $\theta_{jk}^{(l)}$ enters L only via $z_{\text{in},j}^{(l)}$, and $\frac{\partial z_{\text{in},j}^{(l)}}{\partial \theta_{jk}^{(l)}} = z_{\text{out},k}^{(l-1)}$ (see def. above), so **all** gradients follow directly from the error signals:

$$\frac{\partial L}{\partial \theta_{jk}^{(l)}} = \frac{\partial L}{\partial z_{\text{in},j}^{(l)}} \cdot \frac{\partial z_{\text{in},j}^{(l)}}{\partial \theta_{jk}^{(l)}} = \delta_j^{(l)} z_{\text{out},k}^{(l-1)}, \quad \text{in matrix form } \frac{\partial L}{\partial \Theta^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{z}_{\text{out}}^{(l-1)})^\top$$

- Missing piece: computing $\boldsymbol{\delta}^{(l)}$ for all layers $\rightarrow$ next slide



EXCURSUS: BP ERROR SIGNAL RECURSION

- $z_{\text{in},j}^{(l)}$ enters L only via the **next** layer's inputs, which depend on it as

$$z_{\text{in},k}^{(l+1)} = \sum_{j'} \theta_{kj'}^{(l+1)} z_{\text{out},j'}^{(l)} = \sum_{j'} \theta_{kj'}^{(l+1)} \sigma(z_{\text{in},j'}^{(l)}) \Rightarrow \frac{\partial z_{\text{in},k}^{(l+1)}}{\partial z_{\text{in},j}^{(l)}} = \theta_{kj}^{(l+1)} \sigma'(z_{\text{in},j}^{(l)})$$

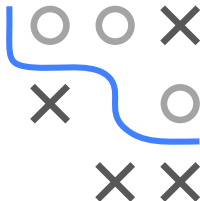
- Chain rule, summing over all units k of layer $l+1$ (= sum over paths), yields the **backward recursion** for $l = m, \dots, 1$:

$$\delta_j^{(l)} = \sum_k \frac{\partial L}{\partial z_{\text{in},k}^{(l+1)}} \cdot \frac{\partial z_{\text{in},k}^{(l+1)}}{\partial z_{\text{in},j}^{(l)}} = \sigma'(z_{\text{in},j}^{(l)}) \sum_k \theta_{kj}^{(l+1)} \delta_k^{(l+1)}$$

$$\text{i.e., } \delta^{(l)} = (\Theta^{(l+1)\top} \delta^{(l+1)}) \odot \sigma'(\mathbf{z}_{\text{in}}^{(l)}) \quad (\odot = \text{elementwise product})$$

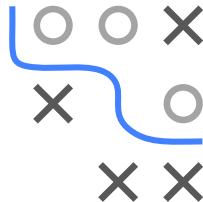
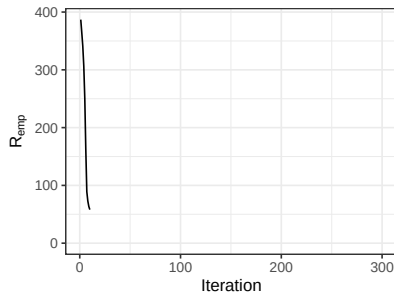
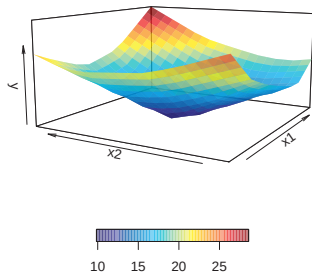
started at $\delta^{(m+1)} = f - y$, reading $\Theta^{(m+1)} = \theta^{(m+1)\top}$ for the output layer

- Together: **forward pass** computes all $\mathbf{z}_{\text{in}}^{(l)}, \mathbf{z}_{\text{out}}^{(l)}$; **backward pass** computes all $\delta^{(l)}$, then all gradients $\delta^{(l)} (\mathbf{z}_{\text{out}}^{(l-1)})^\top$
- Cost: one forward + one backward pass $\Rightarrow$ full gradient costs a small constant multiple ($\approx 2-4\times$) of **one** evaluation of f ; this factor does not grow with the number of weights d (compare finite differences: $d+1$ evaluations)
- To see the factor: forward, backward, and readout $(\Theta^{(l)} \mathbf{z}_{\text{out}}^{(l-1)}), \Theta^{(l+1)\top} \delta^{(l+1)}, \delta^{(l)} (\mathbf{z}_{\text{out}}^{(l-1)})^\top$ each touch every weight once $\Rightarrow \approx 3d$ vs. d multiply-adds for f



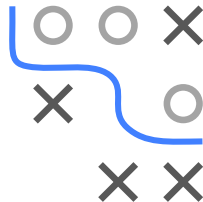
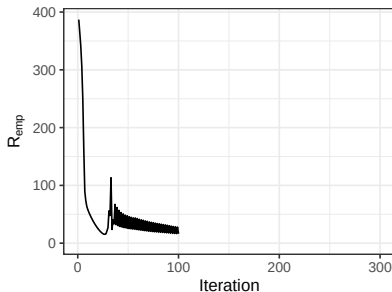
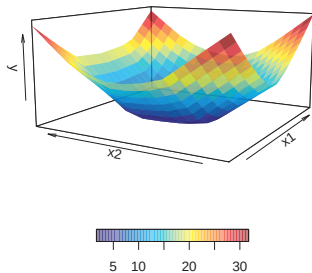
ERM FOR NN WITH GD

After 10 iters of GD:



ERM FOR NN WITH GD

After 100 iters of GD (note the zig-zag pattern of the risk after iter. 30):



ERM FOR NN WITH GD

After 300 iters of GD (note the zig-zag pattern of the risk after iter. 30):

